Breaking Deepfakes With Shared Secrets:

Building The Doppelgänger Protocol

Jonathan Bateman, Founder of `realxreal.ai`

Open Source Cryptography Workshop (OSCW) 2026
Taipei, Taiwan

Roadmap

The Threat Landscape Has Changed

Secure Channels

- Deepfake video → secure zoom call
- Signal group → impersonated individual

Insecure Channels

- Fake account → months-long catfishing over socials
- Voice clone → trusted phone number

The common thread: identity is decided at the human layer

The Identity Binding Problem

Core question: Does this public key really belong to Bob?

Bootstrapping trust is the hard part

- Certificate Authorities
- PGP web of trust
- Trust On First Use (TOFU)
- Phone numbers / SIM
- Email addresses
- OAuth / “Sign in with Google”
- WebAuthn / Passkeys
- Keybase

Why Traditional Authentication Still Fails

- Passwords and 2FA are **transferable**:
  - phishable, replayable, coercible
- Biometrics are **cloneable** with AI:
  - synthetic voice / face can pass “what you are”
- Even secure apps fail when the victim believes the caller is real

Key point

Deepfakes target the human decision point *not the crypto*

Security = Usability

RWC: If users maneuver around it, it is not secure!

Constraint: make defeating deepfakes as easy as sending a text

“Could your grandma use it?”

The Question That Started This

What does every **human have** that no AI can **fake**?

Bridge: “I have their public key” → “I know it′s the right person”

The Twist

Shared Memory Agreement

⇓

Challenge–Response

⇓

Key Exchange

⇓

Verified Trust

The Oldest Authentication Mechanism

Phrasal call-and-response: authenticate via a pre-shared challenge / question Q and expected answer A (meaningless to outsiders)

- **Countersigns**
- **Dead drops / One-time pads (OTPs)**
- **Recognition signals**
- **Duress codes**

Through-line: open channel ⇒ requires private knowledge

Sci-Fi Solved Deepfakes & Doppelgängers

Star Trek

Personal episode prompt
“Who was my Academy roommate?”

Blade Runner

Experience probing
Voight–Kampff: affect + memory cues

The Terminator

Shared codeword
“Say the line only he knows.”

The Core Insight

What does Bob have that a doppelgänger doesn′t?

Cloneable signals

- ★ Voice
- ★ Face

Inaccessible signal

- ★ Shared episodic memory

Private → high-entropy → unforgeable at the human layer

What Is The Doppelgänger Protocol?

- A **memory-gated** verification protocol for deepfake-era communication[1]

Authenticate a person by proving shared experience *without* trusting audio, video, or biometrics

[1] *Doppelgänger* (German): "double walker" — a look-alike or double of a living person

The Naive Approach

"In what city were you born?"

guessable · brittle · gameable

Design requirements

High entropy · **Semantically flexible** · **Experiential**

What Good Memory Challenges Look Like

High-entropy prompts

- “In what cities were our layovers on the way to RWC 2026?”
- “What show do we always argue about?”
- “What did I order on our first date?”
- “What did you say after we got lost in Taipei?”

Shared context

- private
- experiential
- relationship-specific

But how do we turn natural-language answers into something we can test mathematically?

Enter Embedding Models

- **2013**: Tomas Mikolov and his team at Google create **Word2vec**
- An **embedding model** converts text into a vector representing its *meaning* in high-dimensional space
- The vector captures **semantic meaning**, so similar texts are placed close together
- This allows machines to compare language by meaning, not just by exact words

Security Considerations of Embedding Models

Property	**Why it matters**
Dimensionality (d)	Higher $d \Rightarrow$ finer semantic separation; fewer "near-miss" passes
Model consistency	Must match `modelID` / version on both sides, or scores drift
Local-first embeddings	Server-side embedding leaks plaintext answers to a third party

This implementation: all-MiniLM-L6-v2 $\|$ $d = 384$ (semantic fidelity · browser-feasible)

Embeddings → Verification (Cosine Threshold)

Mechanism

1. Embed stored answer A and response $\hat{A}$:

$$A \mapsto e, \quad \hat{A} \mapsto \hat{e} \in \mathbb{R}^d$$

2. Similarity score (semantic match):

$$s = \cos(e, \hat{e}) = \frac{e \cdot \hat{e}}{\|e\| \, \|\hat{e}\|} \in [0, 1]$$

3. Accept iff $s \geq \tau$

No case sensitivity, no exact strings

Toy example

"Park in Taipei" vs "Park in Taiwan" ✓
"Park in Taipei" vs "Parked Car" ×

Security parameter: τ
everyday: $\tau \geq 0.75$
security aware: $\tau \geq 0.85$
high-stakes: $\tau \geq 0.98$

Three Parties, One Goal

ALICE (browser)	SERVER	BOB (browser)
ECDH key generation	Session relay	ECDH key generation
Memory commitment	Similarity gate	Memory response
Key derivation (HKDF)	Key-blind	Key derivation (HKDF)

Goal: Alice and Bob derive the same shared key,
while the server never sees either private key

Step-by-Step Flow (Alice)

1. **ECDH keygen (browser):** $(sk_A, pk_A) \leftarrow \mathsf{KeyGen}()$
   - → sk_A stays local; export pk_A as JWK
2. **Set memory challenge:** choose (Q, A) and compute $e_A \leftarrow \mathsf{Embed}(A)$
3. **Post session state:** `{ alice_pubkey_jwk, answer_embedding }`
   - → Server stores in Redis, returns `session_id` + link / QR
4. **Wait:** open Server-Sent-Events (SSE) channel for `PASS` + Bob′s public key

Step-by-Step Flow (Bob)

5. **Open link:** load `session_id` and view Q
6. **ECDH keygen (browser):** $(sk_B, pk_B) \leftarrow \mathsf{KeyGen}()$
7. **Submit response:** `{ bob_pubkey_jwk, answer }`
8. **Server gate (similarity check):**
   - $e_B \leftarrow \mathsf{Embed}(\hat{A})$, compute $s = \cos(e_A, e_B)$
   - iff $s \geq \tau \Rightarrow$ **PASS**
   - publish SSE: `{ bob_pubkey_jwk, alice_pubkey_jwk }`

The ECDH Handshake

9. Alice receives pk_B via SSE → imports it
10. Alice derives:

$$Z_A \leftarrow \mathsf{ECDH}(sk_A, pk_B) \quad \Rightarrow \quad K_A \leftarrow \mathsf{HKDF\text{-}SHA256}(Z_A) \text{ (AES-GCM 256)}$$

11. Bob receives pk_A → imports it
12. Bob derives:

$$Z_B \leftarrow \mathsf{ECDH}(sk_B, pk_A) \quad \Rightarrow \quad K_B \leftarrow \mathsf{HKDF\text{-}SHA256}(Z_B) \text{ (AES-GCM 256)}$$

$$Z_A = Z_B \quad \Rightarrow \quad K_A = K_B \quad \checkmark \quad \text{(ECDH correctness)}$$

→ Private keys never leave the browsers
→ Server relays public keys only (key-blind)
→ Memory verification gates the entire exchange

What the Server Actually Sees

Data	Seen?	Where / why
Alice private key sk_A	**No**	Never leaves Alice's browser
Bob private key sk_B	**No**	Never leaves Bob's browser
Shared key K	**No**	Derived locally via ECDH + HKDF
Alice plaintext answer A	**Yes (current)**	Forwarded to embedding microservice (future: client-side embed)
Bob plaintext answer $\hat{A}$	**Yes (current)**	Forwarded to embedding microservice (future: client-side embed)
Embeddings (e_A, e_B)	**Yes**	Used for cosine score + threshold gate
Challenge question Q	**Yes**	Display + session context
Public keys (pk_A, pk_B)	**Yes**	Relay-only for handshake via SSE

One-Way Trust Is Half the Job

- Basic flow: Alice challenges Bob
- Question: why should Bob trust “Alice”?
- High-stakes: require mutual challenge (Bob asks Alice a question)

Complete iff $(s_A \geq \tau_A) \wedge (s_B \geq \tau_B)$

Symmetric authentication; no trusted third party

Multi-Question Pools & Random Selection

- Each side preloads $k \in \{2, 3\}$ challenges
- Verify using a random pick: $i \leftarrow \mathsf{Unif}(\{1, \ldots, k\})$
- Attacker must be ready for *all* prompts, not one leaked answer

Security grows with pool size: more prompts $\Rightarrow$ higher attacker cost

Continuous Authentication

- △ Trust decays; device compromise happens *after* onboarding
- △ Re-run a random challenge later: $i \leftarrow \mathsf{Unif}(\{1, \ldots, k\})$
- △ Enforce a response window T (freshness / anti-research)

Pass fast ⇒ proceed **fail / no response** ⇒ escalate

Ongoing verification with no new infrastructure

The Bootstrap Advantage

Time-pressure shift

- defender runs one memory-gated pairing
- attacker must pre-emptively guess the relationship-specific secret

Example

- impersonate journalist *before* pairing
- must answer: *"Where did we first meet?"*
- failure ⇒ bootstrap aborts

Security Considerations

MITM (main risk)

- ✕ link intercepted / forwarded
- ✕ defanged: keys are released only if $s \geq \tau$
- ✕ attacker gets a question, not a session key

Replay (deepfake)

- ✕ fresh session ⇒ fresh challenge
- ✕ time-boxed response window

OSINT guessing

- ✕ use *subjective* / *inside* context
- ✕ MES penalizes low-entropy prompts

Device compromise

- ✕ out of scope: spyware can observe answers
- ✕ mitigate with Secure Enclave + ephemeral sessions

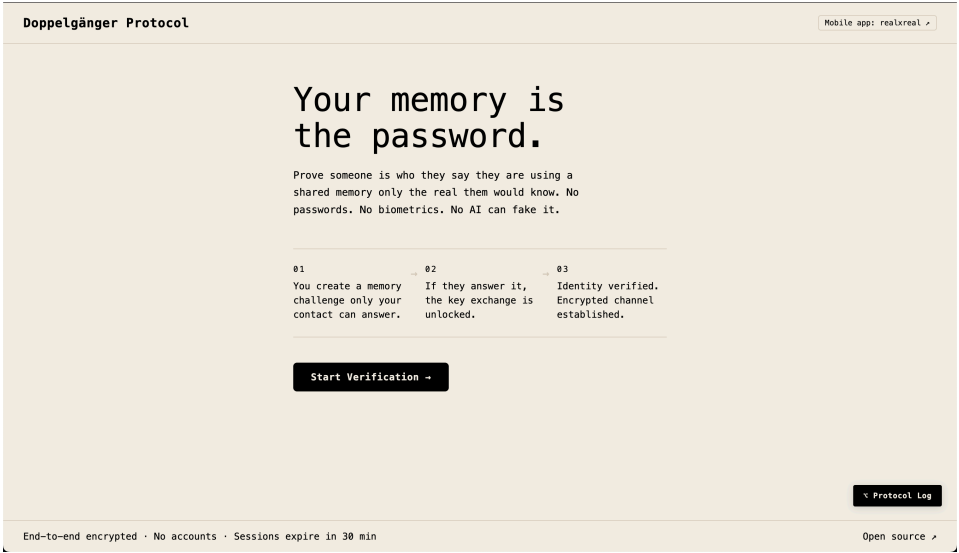
What to Remember

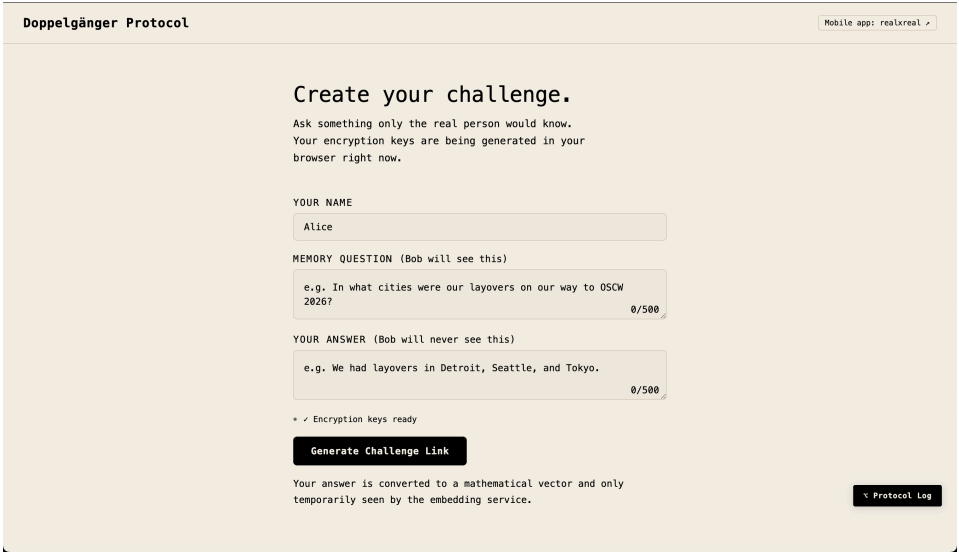
- ✓ Deepfakes break voice / video trust
- ✓ The Doppelgänger Protocol verifies via **human** shared-memory challenges
- ✓ On-device keys, minimal disclosure
- ✓ Bootstrap trust before requesting money, access, or secrets

Bottom line: verify the person, not the media!

What's Next?

- **Memory Entropy Score (MES):** a framework to measure and guide high entropy memory challenges
- **Local embeddings:** run the embedding model in-browser so raw answers never leave the device
- **Protocol spec:** a public Doppelgänger Protocol RFC / whitepaper (primitives, threat model)
- **Open-source iteration:** audit, fork, break, and improve the design


Doppelgänger Protocol
Mobile app: realxreal ↗
Your memory is the password.
Prove someone is who they say they are using a shared memory only the real them would know. No passwords. No biometrics. No AI can fake it.
01
You create a memory challenge only your contact can answer.
02
If they answer it, the key exchange is unlocked.
03
Identity verified. Encrypted channel established.
Start Verification →
⌥ Protocol Lag
End-to-end encrypted · No accounts · Sessions expire in 30 min
Open source ↗


Doppelgänger Protocol
Mobile app: realxreal ↗
Create your challenge.
Ask something only the real person would know.
Your encryption keys are being generated in your
browser right now.
YOUR NAME
Alice
MEMORY QUESTION (Bob will see this)
e.g. In what cities were our layovers on our way to OSCW 2026?
0/500
YOUR ANSWER (Bob will never see this)
e.g. We had layovers in Detroit, Seattle, and Tokyo.
0/500
✓ Encryption keys ready
Generate Challenge Link
Your answer is converted to a mathematical vector and only temporarily seen by the embedding service.
Protocol Log

Doppelgänger Protocol

Mobile app: realxreal ↗

Share this with Bob.

This link contains your memory question and your public key.
Bob must answer correctly before the encrypted channel opens.

http://localhost:5001/verify/s/nHkZiCSXkoQiHwStWA2S… Copy

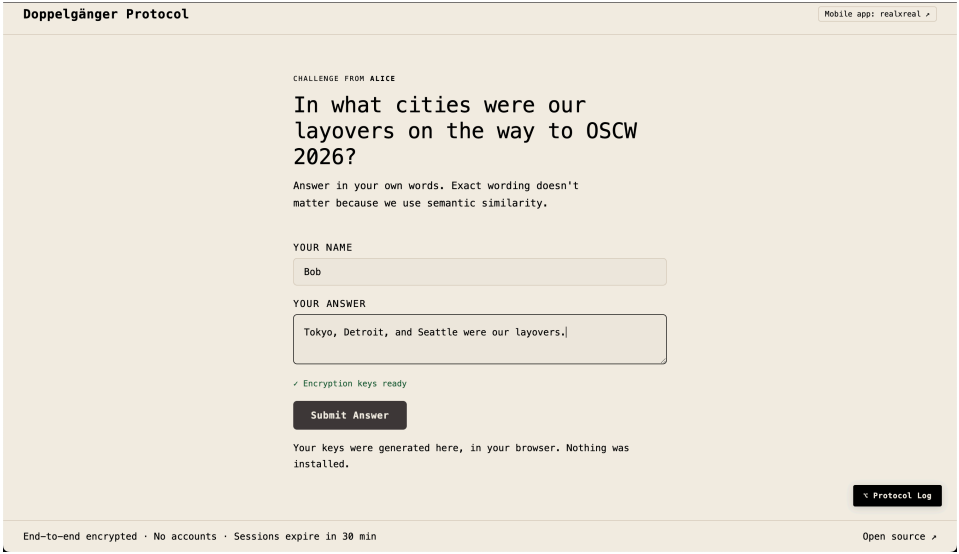
Scan with any camera. This works on any device with a browser.

Waiting for Bob to answer the challenge...

Protocol Log

End-to-end encrypted · No accounts · Sessions expire in 30 min

Open source ↗


Doppelgänger Protocol
Mobile app: realxreal ↗
CHALLENGE FROM ALICE
In what cities were our layovers on the way to OSCW 2026?
Answer in your own words. Exact wording doesn't matter because we use semantic similarity.
YOUR NAME
Bob
YOUR ANSWER
Tokyo, Detroit, and Seattle were our layovers.
✓ Encryption keys ready
Submit Answer
Your keys were generated here, in your browser. Nothing was installed.
⌥ Protocol Log
End-to-end encrypted · No accounts · Sessions expire in 30 min
Open source ↗

Pair up with someone near you. One creates a challenge; the other answers

Acknowledgments

OSCW	Open Source Cryptography Workshop 2026 organizers and attendees
realxreal	Community members and early testers
RIT	Cybersecurity faculty and mentors, including Dr. Billy Brumley (ESL Global Cybersecurity Institute), Prof. Stanislaw Radziszowski (Computer Science, GCCIS), and Angela Srbinovska (incoming PhD student, GCCIS)
OpenBWC	Collaboration team and partners (Rochester Institute of Technology researchers, Rochester Police Department)

Thank you!

Questions?